Assignment no.2

Roman Iakymchuk, TDB, IT Dept
roman.iakymchuk@it.uu.se

April 8, 2026

1 Problem setting

In applications such as computer simulations and digital image processing, an operation that is commonly needed is a *stencil application*. This operation is often performed a large number of times, and the performance of the whole program is highly dependent on it. A *stencil* can be seen as an operator that can be applied on the elements of a vector, matrix or tensor (a "multi-dimensional matrix"). When applying the stencil on an element, you compute a new value using the current value of the element in question, and its neighbours.

As an example, consider the two-dimensional 9 point stencil in Figure 1. Here, each square represents an element in a matrix/a two-dimensional array, while the red dots represent the stencil. When applying the stencil on the element covered by the middlemost dot, we compute a weighted sum of all elements covered by a dot.

In this assignment, we are going to apply a one-dimensional stencil on an array of elements representing function values $f(x)$ for a finite set of N values $x_0, x_1, x_2, ...x_{N-1}$, residing on the interval $0 <= x < 2 \cdot \pi$. Here, each value $x_i = i \cdot h, h = \frac{2 \cdot \pi}{N}$. Applying the stencil on an element v_0 representing the function value $f(x_i)$ simply means computing the sum:

$$\frac{1}{12h} \cdot v_{-2} - \frac{8}{12h} \cdot v_{-1} + 0 \cdot v_0 + \frac{8}{12h} \cdot v_{+1} - \frac{1}{12h} \cdot v_{+2}$$

where v_{-j} is the element j steps to the left of v_0 (that is, $f(x_{i-j})$. Likewise, v_{+j} is the element j steps to the right of v_0 ($f(x_{i+j})$). (Here, the middlemost stencil weight is 0, which is not always the case!) This sum approximates the first derivative $f'(x)$ for $x = x_i$. Your task is to parallelise a short program that applies this stencil on a set of values.

2 Implementation

In Studium, along with these instructions, you can find the (sequential) code that is to be parallelised, a makefile to build the program and a script which is further described in Section 5.

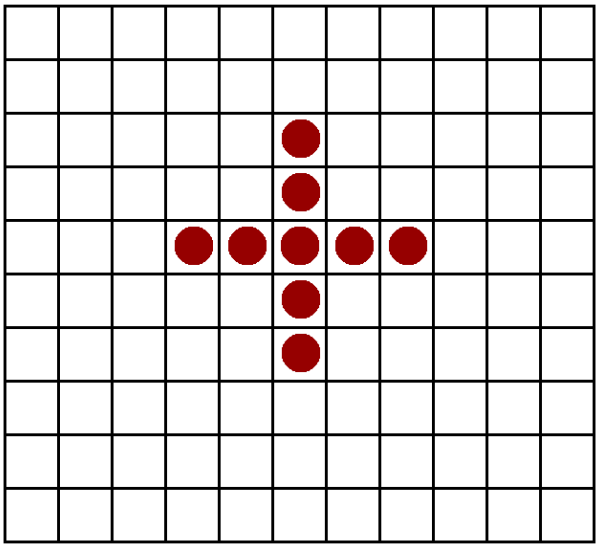

Figure 1: Two-dimensional stencil

2.1 The sequential code

The program provided takes three arguments. The first argument is the name of a file containing the function values on which the stencil will be applied (the input file). The second argument is the name of the file to which the program will write the resulting function values (the output file). The third argument is an integer specifying how many times the stencil will be applied.

The input file must contain $N+1$ numbers. The first number is N (that is, the number of function values) and the N subsequent numbers are the actual function values. The output file is created by the program and will contain the result of the stencil application. (Note that the output file is overwritten if it exists already!)

The program provided reads the input file, applies the stencil the specified number of times and writes the result to the output file. Each stencil application is made on the result of the previous stencil application. The program also prints the number of seconds needed for the stencil application (I/O excluded). The file I/O is handled in the functions `read_input` and `write_output` respectively. There is no reason for you to change these functions. The program applies periodic boundary values. This means that the last element in the array is considered being the closest left neighbour of the first element in the array. Likewise, the first element in the array is used as the closest right neighbour of the last element.

2.2 Requirements for the parallelised code

The I/O of the program should be handled by the process with rank 0. This process is supposed to distribute the values evenly among all processes before the first stencil

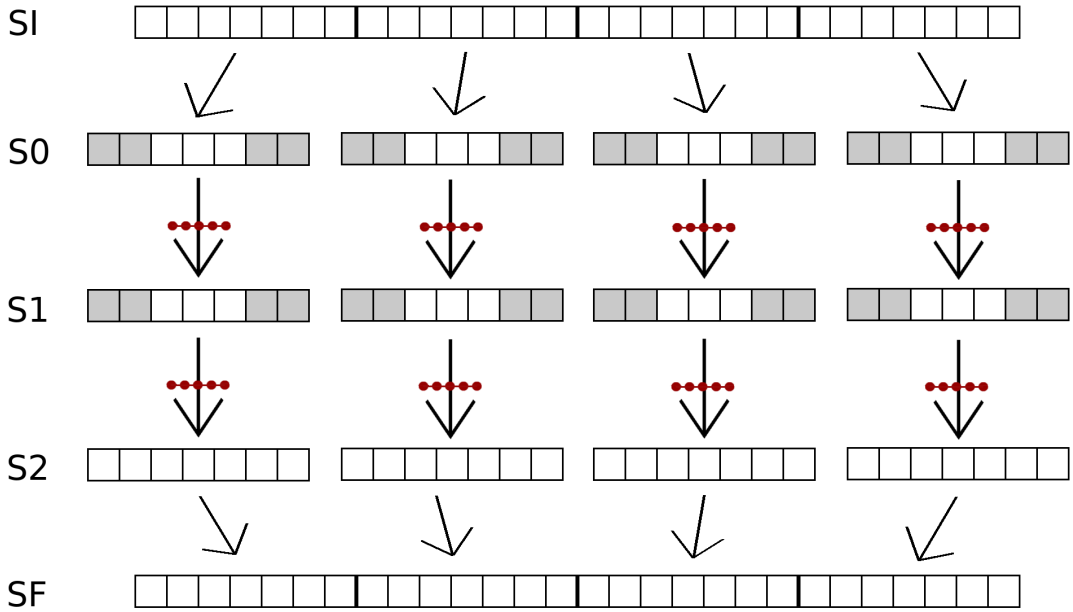

Figure 2: Data distribution and stencil applications. SI is the initial state while SF is the final.

application and collect the result when the last application is done. You may assume that the number of values is divisible by the number of processes. After the initial distribution, each process should apply the stencil on its received values and store the result in another array. Thereafter, each process repeatedly applies the stencil on the result of its last application until the specified number of applications are made. You can assume that the number of function values per process is larger than the stencil width. Note that in order to apply the stencil, each process will need data that is stored on its neighbouring nodes. How will you handle that?

Figure 2 illustrates the procedure when 4 processes are used to apply a stencil twice on an array of length 28. Initially (in state SI), all values are stored in process 0. Next, the values are distributed among the four processes and in state S0 each process keeps its own part (7 elements) of the array. Thereafter, the stencil is applied twice, bringing us to state S2 via state S1. The second stencil application is made on the result of the first one. In state S2, the final result is computed, but it is still distributed among the four processes. Hence, to reach the final state (SF) and being able to report the result, process 0 must collect all the resulting values. The array elements that are marked with grey are needed for the stencil application on a neighbouring node.

Each process is supposed to measure the time to apply the stencil, but only the largest value (among all processes) should be printed. The time measurement shall include the application of the stencil and the inter-process communication that needs to be done in each step. It shall not include the time needed for reading the input file and writing of the

output file. Neither shall it include the time needed for the initial distribution or the final collection of data.

Your parallelisation is supposed to speed up the execution of the program, but you must not change its functionality! The assignment is graded automatically and any difference in the output (except from the value (but not the format!) of the stencil application time), compared to the original program, will lead to immediate rejection. To check the correctness of your program, you may use the smallest files stored in `/proj/uppmax2026-1-92/A2/` on UPPMAX. Files whose name start with `input` contain input data for the program, on the format specified above. The number in the name of the input files specify the number of function values stored in the file. The files named `outputX_Y_ref.txt` can be compared to the output of your program in order to verify the results. The numbers in the file names (`X` and `Y`) specify the number of function values in the file, and the number of times that the stencil has been applied in order to create the output file. You may also plot the input and the output values (eg in R, MATLAB, or Python), see below an example of using matplotlib from Python. Does the output look like the $d : th$ derivative of the input after d stencil applications?

```python
import numpy as np
import matplotlib.pyplot as plt

# Fit y aprox c0 + c1*x with least squares
# https://numpy.org/devdocs/reference/generated/numpy.linalg.lstsq.html
x_data = np.array([0, 1, 2, 3, 4], dtype=float)
y_data = np.array([1.1, 2.0, 2.9, 4.2, 5.1], dtype=float)

# Design matrix for a straight line
A = np.column_stack([np.ones_like(x_data), x_data])
print(A)
coeffs, residuals, rank, s = np.linalg.lstsq(A, y_data, rcond=None)

print('coefficients [c0, c1] =', coeffs)
print('residuals =', residuals)
print('rank =', rank)
print('singular values =', s)

_ = plt.plot(x_data, y_data, 'o', label='Original data', markersize=10)
_ = plt.plot(x_data, coeffs[0] + coeffs[1]*x_data, 'r', label='Fitted line')
_ = plt.legend()
plt.show()
```

3 Performance experiments

When the program is parallelised, it is time for evaluation of its parallel performance. You can run your code on a laptop or on a few MPI processes at Pelle to begin with. However, your code should compile and run on the Linux system. There should be no warnings in the compilation step, when the provided makefile is used.

On the Linux servers, you may use the arguments `--bind-to none` for `mpirun` in order to allow for optimal scheduling of the processes. You are supposed to evaluate the strong and weak scaling of your parallel program. You may use the large input files stored in `/proj/uppmax2026-1-92/A2/` on UPPMAX for your evaluation. For strong scalability, it is enough to experiment with one file size. For the weak scalability experiments you should vary the input size. It is also (a second option) possible to vary the number of stencil applications however the data may be cached making the problem more compute intensive.

Note 1: Run at first on 2 or 4 MPI processes to check correctness and scalability. Then, you can proceed to more processes. You should not use more than two nodes with the number of MPI processes equal to the total number of cores.

Note 2: You have a limited disc quota on the Linux servers and you will not be able to store the input and output files from all runs. Therefore, do not copy the large input files to your home directory. Also remove all large output files immediately! You may use `/scratch/uppmax2026-1-92/YOUR_FOLDER` to store the output files. You may also deactivate writing the output file by commenting out the line

```
#define PRODUCE_OUTPUT_FILE
```

from `stencil.h` while performing the performance experiments. However, it is important that you restore it before submitting the assignment. Otherwise, we will not be able to check your program and your submission will be rejected.

4 Report

You are supposed to write a report on your results – **6-8 pages max** (shorter is better). The report should contain

1. A brief description of your parallelization. Which kind of communication did you use? Why?

2. A description of your performance experiments.

3. The results from the performance experiments. Execution times for different problem sizes and different number of processing elements should be presented in tables. Speedup values should be presented both in a table and a plot. Also plot the ideal speedup in the same figure. The plots can be made using Python, or any other tool that you are familiar with.

4. Discussion of the results. Does the performance of the parallel program follow your expectations? Why/why not?

5 Files to be submitted

The file that you upload in Studium must be a compressed tar file named `A2.tar.gz`. It shall contain a directory named A2, with the following files inside:

- A PDF file named A2_Report.pdf containing you report.
- Your code, following the specifications listed in Section 2. Note that the code must compile without warnings on the Linux system!
- The provided makefile.

Before submitting, please check your file using the script `check_A2.sh` included in `assignment2.tar.gz` (which you have downloaded from Studium):

1. Put `check_A2.sh` and `A2.tar.gz` in the same directory.
2. Create a directory named `test_data` in that directory. Without changing their names, copy the following files from `/proj/uppmax2026-1-92/A2/` on the Linux servers file system to `test_data`:
   - `input96.txt`
   - `output96_1_ref.txt`
   - `output96_4_ref.txt`
3. Make sure that you have execution permission of the script: `chmod u+x check_A2.sh`
4. Run the script: `./check_A2.sh`.
5. If you get the output "`Your file is ready for submission. Well done!`", you may submit the file. Otherwise, fix it according to the output you get.

If you don't get the message "`Your file is ready for submission. Well done!`" your file is **not** ready for submission, and will not be graded until you have fixed the issue(s). However, note that the message is no guarantee for passing the assignment. We will perform additional tests of you code, which we will take into consideration together with your report when we grade the assignment.

Important note: When you upload a first revised version of your work, change the name of the file as `A2-R1.tar.gz` and if a second revision is allowed - `A2-R2.tar.gz`.

Good luck,
Teaching team

Any comments on the assignment will be highly appreciated and will be considered for further improvements. Thank you!